

CURSO BIGQUERY - PROYECTO FINAL

Alumno: Gomez Mendoza Franklin Kevin

Docente: Tabita Pisco Ostos

Fecha: 30/05/2026

Tema: Solución Analítica End-to-End para E-commerce (Ventas, Rentabilidad y Tráfico Web)

1. Definición Estratégica y Caso de Uso

1.1 Objetivo del Proyecto

El objetivo de este proyecto es construir una solución analítica integral en BigQuery para evaluar la salud de un comercio electrónico global. El análisis trasciende el volumen de ventas tradicional, incorporando la rentabilidad operativa (margen por categoría) y el comportamiento del usuario (embudo de conversión web y canales de adquisición), proporcionando una visión del negocio en un dashboard interactivo de tres dimensiones: Ejecutiva, Producto y Marketing.

1.2 Caso de Uso y Necesidad de Negocio

- **Problema a resolver:** La dirección del e-commerce requiere visibilidad integrada sobre el rendimiento por país, las categorías que realmente generan margen de ganancia y los cuellos de botella en la navegación web, para tomar decisiones informadas sobre la asignación de recursos.
- **Público Objetivo:** El reporte interactivo está diseñado para tres perfiles clave:

Perfil	Página	Necesidad Principal
Director General	Página 1	Supervisión de KPIs financieros, crecimiento global y distribución geográfica de ingresos.
Gerente de Operaciones / Catálogo	Página 2	Evaluación de rentabilidad por categoría de producto y optimización del catálogo por país.
Gerente de Marketing	Página 3	Análisis de efectividad de canales de adquisición y diagnóstico del embudo de conversión web.

- **Métricas Clave:**
 - Ingresos Totales, Margen Total, Total de Órdenes
 - Margen (%), Crecimiento Mes a Mes (MoM) — calculado con `LAG()`
 - Total de Sesiones y Clientes por Fuente de Tráfico

- **Accionabilidad:**
 - Optimizar el catálogo priorizando las categorías con mayor margen (ej. *Outerwear & Coats*).
 - Reasignar el presupuesto de marketing hacia las fuentes de tráfico más efectivas (ej. *Email* y *Adwords*).
 - Identificar áreas de mejora en la experiencia de usuario (UX) al analizar el alto volumen de sesiones en "3_Producto" que no avanzan al carrito.

2. Origen de Datos y Arquitectura

Se seleccionó la base de datos pública `bigquery-public-data.thelook_ecommerce`, extrayendo **6 tablas interrelacionadas** que cubren el espectro demográfico, transaccional y de navegación: `users`, `orders`, `order_items`, `products`, `inventory_items` y `events`.

Tabla Fuente	Descripción y Rol en el Modelo
<code>users</code>	Datos demográficos del cliente: edad, país, género y fuente de adquisición.
<code>orders</code>	Cabecera de cada pedido: fecha de creación, estado y ID de usuario.
<code>order_items</code>	Detalle de cada línea de pedido: precio de venta, producto e inventario asociado.
<code>products</code>	Catálogo de productos: nombre, categoría, departamento y precio de lista.
<code>inventory_items</code>	Registro de costos de cada ítem de inventario (clave para el cálculo del margen).
<code>events</code>	Log de eventos de navegación web: tipo de evento, sesión, usuario y fuente de tráfico.

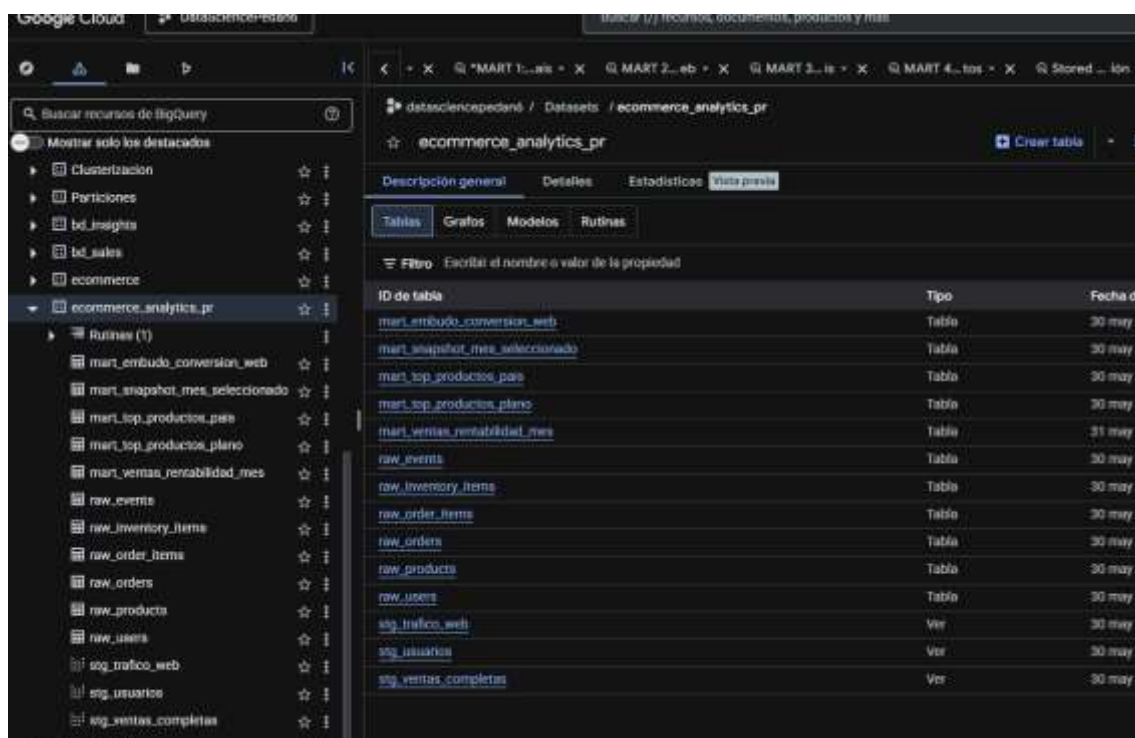


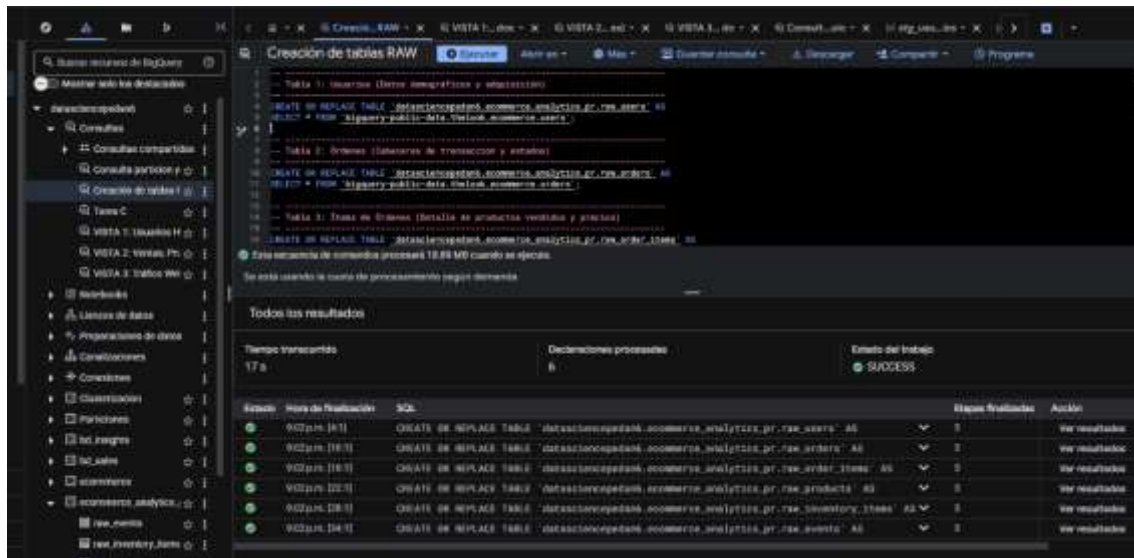
Figura 1: Evidencia de la creación del entorno de trabajo en BigQuery. Confirma la existencia del dataset en la región US y el prefijo raw_ en las 6 tablas fuente, demostrando el aislamiento correcto de la capa de extracción.

3. Modelo de Datos y Fases de Transformación

La solución fue modelada siguiendo las mejores prácticas de ingeniería de datos, dividiendo el procesamiento en tres capas claras: RAW, STG (Staging) y MARTS.

3.1 Capa RAW (Extracción Controlada)

Se ejecutaron copias 1:1 de las fuentes públicas para congelar el estado de los datos, evitar consultas repetitivas a la base pública (control de costos) y aislar nuestra lógica de negocio de posibles cambios en el origen.



raw_events	☆	⋮
raw_inventory_items	☆	⋮
raw_order_items	☆	⋮
raw_orders	☆	⋮
raw_products	☆	⋮
raw_users	☆	⋮

Figura 2: Evidencia de la Capa RAW. Valida que las 6 tablas fueron extraídas y almacenadas correctamente en el dataset propio, desacoplando la solución de la fuente pública.

3.2 Capa STG (Transformación, Limpieza y JOINS Múltiples)

En esta capa intermedia, se realizaron las traducciones de dimensiones al español ("hispanización"), la limpieza de registros nulos y la unificación de entidades. Se destaca el cálculo temprano del "Margen de Ganancia" a nivel de ítem ($\text{precio_venta} - \text{costo_producto}$).

Para cumplir con la integración relacional, se utilizaron múltiples tipos de JOINS en la vista `stg_ventas_completas`:

- **INNER JOIN:** Para cruzar `order_items` con `orders` y `products`, asegurando que solo se analicen productos del catálogo actual con una orden válida asociada.

- **LEFT JOIN:** Para cruzar el detalle de venta con inventory_items y obtener el costo, garantizando que, si un producto carece de registro de costo, la venta no se pierda del análisis general.

```
-- VISTA 1: Usuarios Hispanizados
-- Limpieza de géneros y selección de columnas clave.
CREATE OR REPLACE VIEW `TU_PROYECTO.ecommerce_analytics_pr.stg_usuarios`
AS
SELECT
  id AS id_usuario,
  first_name AS nombre,
  last_name AS apellido,
  age AS edad,
  -- Estandarización y traducción de género
  CASE
    WHEN gender = 'M' THEN 'Masculino'
    WHEN gender = 'F' THEN 'Femenino'
    ELSE gender
  END AS genero,
  country AS pais,
  traffic_source AS fuente_adquisicion,
  DATE(created_at) AS fecha_registro
FROM `TU_PROYECTO.ecommerce_analytics_pr.raw_users`;
```

```
-- VISTA 2: Ventas, Productos y Rentabilidad (JOINS múltiples)
-- INNER JOIN y LEFT JOIN.
-- Además calculamos el Margen de Ganancia.
CREATE OR REPLACE VIEW
`TU_PROYECTO.ecommerce_analytics_pr.stg_ventas_completas` AS
SELECT
  oi.id AS id_detalle,
  o.order_id AS id_orden,
  o.user_id AS id_usuario,
  DATE(o.created_at) AS fecha_orden,
  FORMAT_DATE('%Y-%m', o.created_at) AS mes_orden,
  -- Traducción de estados de la orden
  CASE o.status
    WHEN 'Shipped' THEN 'Enviado'
    WHEN 'Complete' THEN 'Completado'
    WHEN 'Processing' THEN 'En Proceso'
    WHEN 'Cancelled' THEN 'Cancelado'
    WHEN 'Returned' THEN 'Devuelto'
    ELSE o.status
  END AS estado_orden,
```

```

p.category AS categoria_producto,
p.department AS departamento_producto,
oi.sale_price AS precio_venta,
ii.cost AS costo_producto,
-- Creación de métrica de negocio clave:
(oi.sale_price - ii.cost) AS margen_ganancia
FROM `TU_PROYECTO.ecommerce_analytics_pr.raw_order_items` oi
INNER JOIN `TU_PROYECTO.ecommerce_analytics_pr.raw_orders` o
    ON oi.order_id = o.order_id
-- INNER JOIN: Solo queremos ítems que tengan un producto asociado en el
catálogo
INNER JOIN `TU_PROYECTO.ecommerce_analytics_pr.raw_products` p
    ON oi.product_id = p.id
-- LEFT JOIN: Traemos el costo del inventario (si un ítem no tiene
inventario mapeado,
-- no queremos perder la venta, solo su costo quedará en null)
LEFT JOIN `TU_PROYECTO.ecommerce_analytics_pr.raw_inventory_items` ii
    ON oi.inventory_item_id = ii.id;

```

Figura 3: Evidencia directa del requisito de JOINS múltiples de la rúbrica. Muestra ambos tipos de JOIN con sus respectivas condiciones ON y la columna margen_ganancia calculada.

```

-- VISTA 3: Tráfico Web y Funnel
-- Clasificamos los eventos numéricamente para hacer el embudo en Looker
Studio.
CREATE OR REPLACE VIEW
`TU_PROYECTO.ecommerce_analytics_pr.stg_trafico_web` AS
SELECT
    id AS id_evento,
    user_id AS id_usuario,
    session_id AS id_sesion,
    DATE(created_at) AS fecha_evento,
    FORMAT_DATE('%Y-%m', created_at) AS mes_evento,
    traffic_source AS fuente_trafico,
    -- Clasificación para ordenar el embudo de conversión lógicamente
    CASE event_type
        WHEN 'home' THEN '1_Inicio'
        WHEN 'department' THEN '2_Departamento'
        WHEN 'product' THEN '3_Producto'
        WHEN 'cart' THEN '4_Carrito'
        WHEN 'purchase' THEN '5_Compra'
        WHEN 'cancel' THEN '6_Cancelación'
        ELSE event_type
    END AS tipo_evento_funnel
FROM `TU_PROYECTO.ecommerce_analytics_pr.raw_events`;

```

Capturas del código SQL de la Capa STG, enfocándose en la parte donde se ven los comandos INNER JOIN y LEFT JOIN]

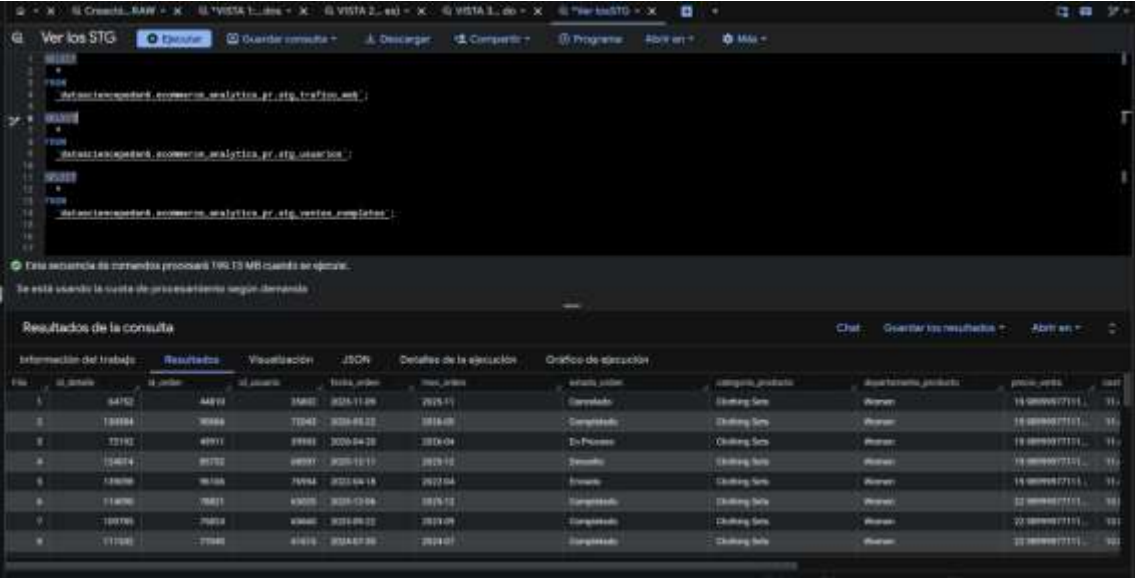


Figura 4: Evidencia de los datos transformados. Confirma la correcta hispanización de los estados, el formato de fecha y la nueva métrica de negocio calculada (margen_ganancia).

4. Estructuras Complejas y Analítica Avanzada (ARRAY/STRUCT y UNNEST)

La Capa MARTS fue diseñada para generar las tablas finales (agregadas) que alimentan directamente a la herramienta de visualización. En esta etapa se aplicaron funciones complejas de Google SQL para optimizar el rendimiento y agregar valor analítico.

4.1 MART 1: Rendimiento Comercial y Cálculo MoM con LAG()

La tabla `mart_ventas_rentabilidad_mes` agrega los indicadores financieros clave por mes y país. El valor diferencial de esta tabla es la implementación de la función analítica `LAG()`, que permite acceder al valor del período anterior dentro de una ventana de datos particionada, habilitando el cálculo nativo del crecimiento Mes a Mes (MoM).

Criterio de Rúbrica: Se utiliza la función analítica `LAG()` con `PARTITION BY` y `ORDER BY` para calcular el crecimiento MoM sin joins adicionales.

```

-- MART 1: Rendimiento Comercial y Rentabilidad Mensual por País
-- Uso de función analítica LAG() para comparar vs mes anterior.
CREATE OR REPLACE TABLE
`TU_PROYECTO.ecommerce_analytics_pr.mart_ventas_rentabilidad_mes` AS
WITH metricas_base AS (
    SELECT
        v.mes_orden,
        u.pais,
        COUNT(DISTINCT v.id_orden) AS total_ordenes,
        COUNT(DISTINCT v.id_usuario) AS total_clientes,
        ROUND(SUM(v.precio_venta), 2) AS ingresos_totales,
        ROUND(SUM(v.costo_producto), 2) AS costo_total,
        ROUND(SUM(v.margen_ganancia), 2) AS margen_total
    FROM `TU_PROYECTO.ecommerce_analytics_pr.stg_ventas_completas` v
    LEFT JOIN `TU_PROYECTO.ecommerce_analytics_pr.stg_usuarios` u
        ON v.id_usuario = u.id_usuario
    WHERE v.estado_orden NOT IN ('Cancelado', 'Devuelto') -- Solo ventas
efectivas
    GROUP BY 1, 2
)
SELECT
    *,
    -- Calculamos el porcentaje de rentabilidad (% de Margen)
    ROUND(SAFE_DIVIDE(margen_total, ingresos_totales) * 100, 2) AS
margen_porcentaje,
    -- Función LAG: Traemos los ingresos del mes anterior para el mismo
país
    LAG(ingresos_totales) OVER(PARTITION BY pais ORDER BY mes_orden) AS
ingresos_mes_anterior,
    -- Cálculo del crecimiento MoM (Month-over-Month)
    ROUND(SAFE_DIVIDE(
        ingresos_totales - LAG(ingresos_totales) OVER(PARTITION BY pais
ORDER BY mes_orden),
        LAG(ingresos_totales) OVER(PARTITION BY pais ORDER BY mes_orden)
    ) * 100, 2) AS crecimiento_ingresos_pct
FROM metricas_base;

```

Figura 5: Evidencia del uso de LAG(). Muestra cómo la tabla materializada contiene el valor del mes anterior en la misma fila, confirmando el cálculo correcto del crecimiento MoM.

4.2 MART 2: Embudo de Conversión Web

La tabla **mart_embudo_conversion_web** agrega las sesiones y usuarios únicos por etapa del funnel y fuente de tráfico, generando el insumo directo para el gráfico de embudo en la Página 3 del dashboard.


```
-- MART 2: Embudo de Conversión Web (Funnel)
-- Agregamos el tráfico web para analizar caídas en Looker Studio.
CREATE OR REPLACE TABLE
`TU_PROYECTO.ecommerce_analytics_pr.mart_embudo_conversion_web` AS
SELECT
    mes_evento,
    fuente_trafico,
    tipo_evento_funnel,
    COUNT(DISTINCT id_sesion) AS total_sesiones,
    COUNT(DISTINCT id_usuario) AS usuarios_unicos
FROM `TU_PROYECTO.ecommerce_analytics_pr.stg_trafico_web`
GROUP BY 1, 2, 3;
```

4.3 MART 3: Top Productos por País con ARRAY_AGG y STRUCT

Esta tabla implementa la estructura de datos anidada más sofisticada del proyecto. Mediante la combinación de la función de ranking **RANK()** con la función de agregación **ARRAY_AGG(STRUCT(...))**, se empaqueta el Top 5 de categorías más rentables de cada país en **una sola celda estructurada**. Esta técnica optimiza el almacenamiento y el rendimiento de consulta al eliminar la explosión de filas por ranking.

Criterio de Rúbrica: Se utilizan ARRAY_AGG y STRUCT para crear estructuras de datos anidadas (tipo RECORD REPEATED en BigQuery).

```
-- -----
-- MART 3: Top Productos por País (Estructura Anidada)
-- Uso de ARRAY_AGG(STRUCT(...)) y función RANK().
-- -----
-- -----

CREATE OR REPLACE TABLE
`TU_PROYECTO.ecommerce_analytics_pr.mart_top_productos_pais` AS
WITH rank_productos AS (
    SELECT
        u.pais,
        v.categoria_producto,
        ROUND(SUM(v.margen_ganancia), 2) AS margen_generado,
        -- Asignamos un ranking basado en el margen de ganancia
        RANK() OVER(PARTITION BY u.pais ORDER BY SUM(v.margen_ganancia)
DESC) AS ranking
    FROM `TU_PROYECTO.ecommerce_analytics_pr.stg_ventas_completas` v
    JOIN `TU_PROYECTO.ecommerce_analytics_pr.stg_usuarios` u
```

```

        ON v.id_usuario = u.id_usuario
    WHERE v.estado_orden NOT IN ('Cancelado', 'Devuelto')
    GROUP BY 1, 2
)
SELECT
    pais,
    -- Empaquetamos el top 5 en un Arreglo de Estructuras (ARRAY/STRUCT)
    ARRAY_AGG(
        STRUCT(ranking, categoria_producto, margen_generado)
        ORDER BY ranking LIMIT 5
    ) AS top_5_productos_rentables
FROM rank_productos
WHERE ranking <= 5
GROUP BY 1;

```

The screenshot shows the Google Cloud BigQuery interface. On the left, the 'e-commerce_analytics_pr' dataset is expanded, showing the 'mart_top_productos_pais' table selected. The main panel displays the table's schema with the following columns:

Nombre del campo	Tipo	Modo	Descripción
pais	STRING	NULLABLE	-
top_5_productos_rentables	RECORD	REPEATED	-
ranking	INTEGER	NULLABLE	-
categoria_producto	STRING	NULLABLE	-
margen_generado	FLOAT	NULLABLE	-

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas
Fila	pais	top_5_produ...	top_5_productos_rentables. cat...	top_5_produ...	
1	Brasil	1	Outerwear & Coats	77433.02	
		2	Jeans	60353.48	
		3	Sweaters	47627.11	
		4	Suits & Sport Coats	44628.66	
		5	Swim	36612.98	
2	Spain	1	Outerwear & Coats	21467.66	
		2	Jeans	16490.76	
		3	Sweaters	14410.44	
		4	Suits & Sport Coats	12752.26	
		5	Swim	9427.48	
3	Japan	1	Outerwear & Coats	14475.31	
		2	Jeans	11226.58	
		3	Sweaters	8374.02	
		4	Suits & Sport Coats	7083.15	
		5	Sleep & Lounge	5848.61	

Figura 6: Evidencia crítica del uso de ARRAY/STRUCT. El esquema de BigQuery debe mostrar la columna `top_5_productos_rentables` con tipo `RECORD` y modo `REPEATED`, confirmando la correcta creación de la estructura anidada

4.4 MART 4: Aplanado de Estructuras con UNNEST()

Para garantizar la compatibilidad con herramientas de BI que requieren esquemas planos (tablas relacionales estándar), se construyó la tabla **`mart_top_productos_plano`** mediante la aplicación explícita de la función **`UNNEST()`** sobre el array de estructuras generado en el MART 3. Este proceso expande cada elemento del array anidado en una fila independiente.

Criterio de Rúbrica: Se utiliza `UNNEST()` para "aplanar" (desanidar) el campo `RECORD REPEATED` a un esquema tabular estándar, compatible con Looker Studio.

```
-- -----
--
-- MART 4: Top Productos
-- Demostración explícita del uso de UNNEST().
-- -----
--
CREATE OR REPLACE TABLE
`TU_PROYECTO.ecommerce_analytics_pr.mart_top_productos_plano` AS
SELECT
  t.pais,
  productos.ranking,
```

```

    productos.categoria_producto,
    productos.margen_generado
FROM `TU_PROYECTO.ecommerce_analytics_pr.mart_top_productos_pais` t,
UNNEST(t.top_5_productos_rentables) AS productos;

```

Fila	pais	ranking	categoria_producto	margen_gen...
1	Belgium	5	Fashion Hoodies & Sweatshirts	2802.72
2	Germany	1	Outerwear & Coats	24923.94
3	United Kingdom	4	Suits & Sport Coats	13041.39
4	Belgium	2	Jeans	5295.81
5	Brazil	3	Sweaters	47627.11
6	Spain	2	Jeans	16490.76
7	Japan	1	Outerwear & Coats	14475.31
8	Germany	4	Suits & Sport Coats	11843.55
9	Spain	1	Outerwear & Coats	21467.66
10	Japan	4	Suits & Sport Coats	7083.15
11	Colombia	2	Jeans	183.04
12	Austria	1	Jeans	73.32
13	Poland	1	Sweaters	1099.48
14	France	4	Suits & Sport Coats	14751.17
15	United States	1	Outerwear & Coats	126787.79
16	France	2	Jeans	22218.05
17	Spain	4	Suits & Sport Coats	12752.26
18	Brazil	4	Suits & Sport Coats	44628.66
19	Austria	4	Socks	12.82
20	Australia	3	Suits & Sport Coats	7575.47

Figura 7: Evidencia del uso de UNNEST(). La tabla muestra una fila por categoría por país, confirmando que el array fue correctamente expandido a un esquema relacional plano.

5. Automatización y Scripting con Stored Procedure

Para dotar al proyecto de **escalabilidad operativa y automatización**, se desarrolló el Stored Procedure **sp_resumen_mensual**. Este procedimiento encapsula la lógica de recarga mensual, permitiendo que cualquier usuario autorizado ejecute una actualización completa del snapshot de datos con un solo comando **CALL**.

5.1 Componentes del Stored Procedure

El procedimiento implementa los siguientes elementos de scripting avanzado:

- **DECLARE:** Instancia variables de control (**var_mes_analisis** de tipo STRING e **var_filas_afectadas** de tipo INT64).

- **Lógica Condicional (IF/ELSE):** Si el parámetro de entrada es NULL o vacío, el procedimiento asume automáticamente el mes vigente mediante **FORMAT_DATE('%Y-%m', CURRENT_DATE())**.
- **EXECUTE IMMEDIATE:** Permite la construcción y ejecución dinámica de sentencias DDL/DML, inyectando la variable **var_mes_analisis** en el cuerpo de la consulta en tiempo de ejecución.
- **@@row_count:** Variable de sistema de BigQuery utilizada para capturar el número de filas materializadas en la tabla de snapshot, proporcionando retroalimentación de ejecución al usuario.

Criterio de Rúbrica: Se emplean DECLARE, SET, IF/ELSE, EXECUTE IMMEDIATE y @@row_count, cumpliendo el criterio de Scripting de la rúbrica.

```
CREATE OR REPLACE PROCEDURE
`TU_PROYECTO.ecommerce_analytics_pr.sp_resumen_mensual` (mes_parametro
STRING)
BEGIN
  -- Declaración de variables (DECLARE)
  DECLARE var_mes_analisis STRING;
  DECLARE var_filas_afectadas INT64;

  -- Lógica de control de fechas: Si el usuario no pasa un mes, usamos
  el mes actual.
  IF mes_parametro IS NULL OR mes_parametro = '' THEN
    SET var_mes_analisis = FORMAT_DATE('%Y-%m', CURRENT_DATE());
  ELSE
    SET var_mes_analisis = mes_parametro;
  END IF;

  -- Usamos EXECUTE IMMEDIATE porque estamos inyectando una variable
  dinámica en el nombre del mes.
  EXECUTE IMMEDIATE FORMAT("""
    CREATE OR REPLACE TABLE
    `TU_PROYECTO.ecommerce_analytics_pr.mart_snapshot_mes_seleccionado` AS
    SELECT
      v.mes_orden,
      v.estado_orden,
      COUNT(DISTINCT v.id_orden) AS total_ordenes,
      ROUND(SUM(v.margen_ganancia), 2) AS margen_total_generado
    FROM `TU_PROYECTO.ecommerce_analytics_pr.stg_ventas_completas` v
    WHERE v.mes_orden = '%s'
    GROUP BY 1, 2;
    """, var_mes_analisis);
```

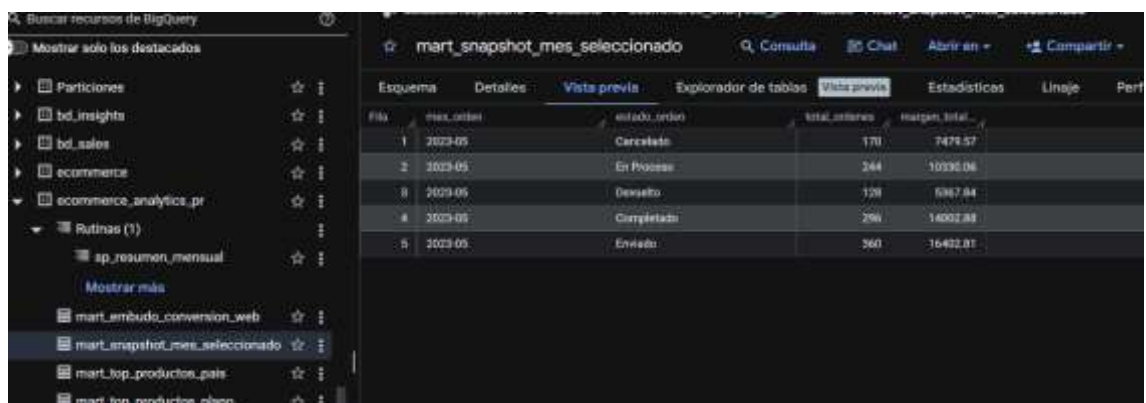
```
-- Capturamos cuántas filas se crearon en la tabla usando una
variable de sistema de BQ
SET var_filas_afectadas = @@row_count;

-- Imprimimos un mensaje de éxito para que el usuario sepa qué pasó
SELECT FORMAT('Éxito. Se calculó el periodo %. Filas generadas en el
resumen: %d', var_mes_analisis, var_filas_afectadas) AS
resultado_ejecucion;

END;
```

```
-- Ejecución del SP

CALL `datasciencepedan6.ecommerce_analytics_pr.sp_resumen_mensual`('2023-
05');
```



File	Max. orden	estado.orden	total_ordenes	Margen. total
1	2023-05	Cancelado	170	7479.57
2	2023-05	En Proceso	244	10392.06
3	2023-05	Devuelto	128	5367.84
4	2023-05	Completado	296	14002.88
5	2023-05	Enviado	360	16422.01

Figura 8: Evidencia de la ejecución del Stored Procedure. Debe mostrar el comando CALL ejecutado sin errores y el mensaje de resultado con el período procesado y el número de filas generadas, validando el uso de DECLARE y @@row_count.

6. Visualización de Alto Impacto (Dashboard en Looker Studio)

El flujo de datos de la solución culmina en Looker Studio, donde se materializa el Caso de Uso a través de un **dashboard interactivo de tres páginas**. Los filtros de *Año-Mes* y *País* fueron configurados a nivel de informe para garantizar una navegación coherente y sincronizada entre las tres vistas.

6.1 Página 1: Visión Ejecutiva

Proporciona la vista macro del rendimiento financiero global. Está diseñada para el **Director General** y responde a la pregunta: ¿Cómo está el negocio en términos generales? Incluye:

- KPIs globales de portada: Ingresos Totales, Margen Total y Total de Órdenes.
- Mapa geoespacial de ingresos por país para identificar los mercados más importantes.
- Tabla de ranking de países por ingresos y margen.
- Gráfico de líneas con la tendencia histórica mensual y el crecimiento MoM (calculado con **LAG()**).

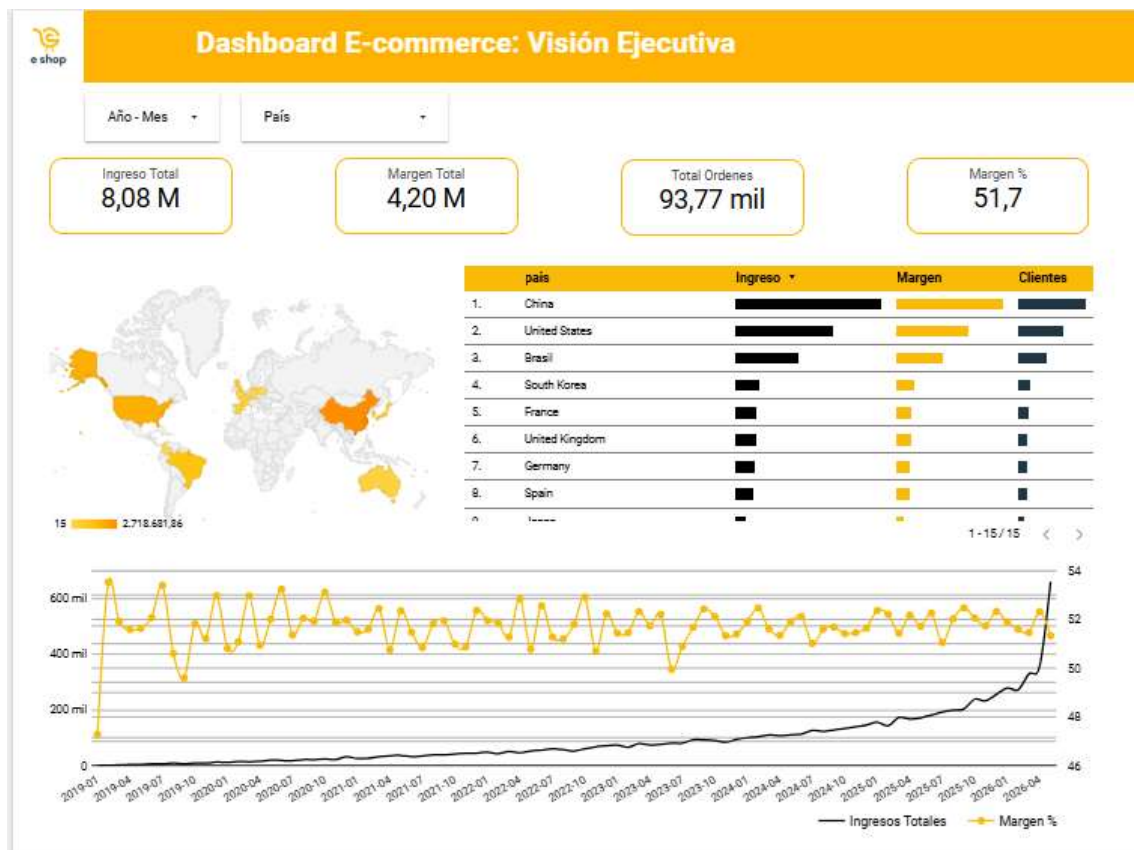


Figura 9: Evidencia del dashboard ejecutivo. Muestra los KPIs financieros globales, la distribución geográfica de ingresos y la tendencia de crecimiento MoM, demostrando la conexión directa con la tabla `mar_venta_rentabilidad_mes`

6.2 Página 2: Análisis de Productos

Desglosa la rentabilidad por categoría de producto y mercado. Diseñada para el **Gerente de Operaciones / Catálogo**, responde a: ¿Qué categorías generan mayor margen real? Incluye:

- Gráfico de barras apiladas al 100% del margen por categoría, para comparar la composición entre países.
- Matriz de calor de margen (%) por categoría vs. período para identificar tendencias de rentabilidad.

- Los datos de *Outerwear & Coats* y *Jeans* se destacan consistentemente como las categorías líderes en margen.

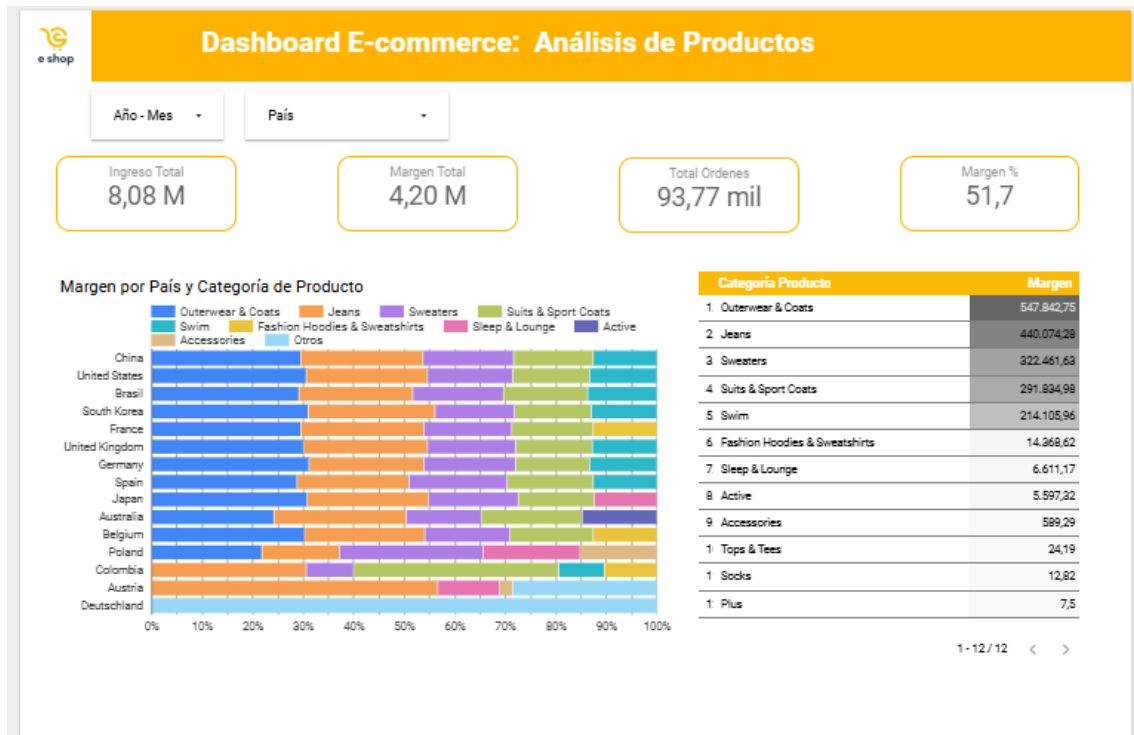


Figura 10: Evidencia del análisis de rentabilidad. Muestra el desglose de margen por categoría de producto, conectando los datos de `mart_top_productos_plano` (generado con `ARRAY_AGG + UNNEST`) con la visualización final.

6.3 Página 3: Marketing & Embudo Web

Evalúa la eficiencia de la adquisición y retención de usuarios. Diseñada para el **Gerente de Marketing**, responde a: ¿Dónde se pierden los usuarios en el camino hacia la compra? Incluye:

- Gráfico de embudo (funnel) de conversión web con las 6 etapas clasificadas numéricamente (1_Inicio → 5_Compra).
- Gráfico de distribución de clientes por canal de adquisición (fuente de tráfico), destacando al *Email* como canal dominante.
- Insight accionable clave: el alto volumen de sesiones que alcanzan **"3_Producto"** sin avanzar al carrito indica una oportunidad de mejora en la presentación de producto (UX).

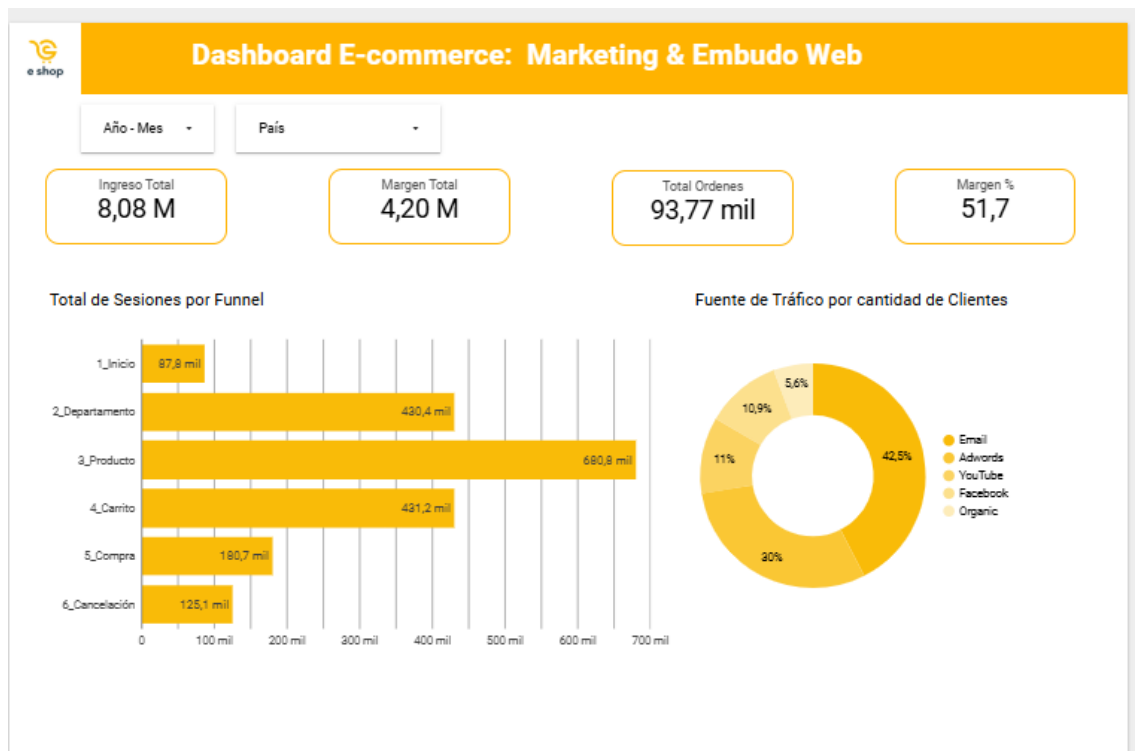


Figura 11: Evidencia del análisis de marketing y conversión. Muestra el embudo de conversión (construido con `mart_embudo_conversion_web`) y la distribución de fuentes de tráfico, cerrando el ciclo analítico end-to-end del proyecto

clave: el alto volumen de sesiones que alcanzan **"3_Producto"** sin avanzar al carrito indica una oportunidad de mejora en la presentación de producto (UX).

7. Conclusiones y Valor Generado

Este proyecto demuestra la implementación de una solución de analítica de datos de nivel productivo, cubriendo íntegramente el ciclo de vida del dato desde la extracción hasta la visualización ejecutiva.

Cumplimiento de Requisitos Técnicos

Criterio de Rúbrica	Elemento Implementado	Sección del Informe
JOINS Múltiples	INNER JOIN + LEFT JOIN	Sección 3.2 — <code>stg_ventas_completas</code>
Función Analítica	LAG() con PARTITION BY	Sección 4.1 — <code>mart_ventas_rentabilidad_mes</code>

Función Analítica	RANK() con PARTITION BY	Sección 4.3 — mart_top_productos_pais
ARRAY / STRUCT	ARRAY_AGG(STRUCT(...))	Sección 4.3 — mart_top_productos_pais
UNNEST	UNNEST() sobre RECORD REPEATED	Sección 4.4 — mart_top_productos_plano
Scripting (DECLARE)	DECLARE, SET, IF/ELSE, EXECUTE IMMEDIATE	Sección 5 — sp_resumen_mensual
Dashboard	3 páginas en Looker Studio con filtros	Sección 6 — Visualización